

LAPORAN PRATIUM STRUKTUR DATA

“ALGORITMA SORTING”



OLEH:

KHAULA LATHIFA FIRDAUSYI

2511531007

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU : Dr. WAHYUDI,
S.T, M.T

ASISTEN PRAKTIKUM : M. GALID A.

FAKULTAS TEKNOLOGI
INFORMASI DEPARTEMEN
INFORMATIKA
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum Struktur Data ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu tugas praktikum sekaligus sebagai bentuk penerapan materi yang telah dipelajari.

Dalam penyusunan laporan ini, penulis memperoleh banyak pengetahuan dan pengalaman, khususnya dalam memahami konsep struktur data Algoritma Sorting serta penerapannya dalam pemrograman Java. Praktikum ini memberikan gambaran mengenai bagaimana teori yang telah dipelajari dapat diimplementasikan secara langsung dalam bentuk program sederhana.

Penulis menyadari bahwa laporan ini masih belum sempurna. Oleh karena itu, penulis mengharapkan saran yang dapat membantu dalam penyempurnaan laporan di masa mendatang.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	i
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	1
BAB II PEMBAHASAN	2
2.1 Dasar Teori.....	2
2.2 Penjelasan Kode Program	3
2.2.1 Class ShellSort_2511531007.....	3
2.2.2 Class QuickSort_2511531007	4
2.2.3 Class MergeSort_2511531007	7
2.2.4 Class BubbleSortGUI_2511531007	9
2.3 Tugas Praktikum	13
2.3.1 Class Lagu_2511531007	13
2.3.2 Class Sorting_2511531007.....	14
BAB III PENUTUP	18
3.1 Kesimpulan	18
3.2 Saran	18
DAFTAR PUSTAKA.....	19

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data dan algoritma merupakan bagian penting dalam pemrograman yang digunakan untuk mengelola serta mengolah data secara efisien. Salah satu algoritma yang sering digunakan adalah algoritma sorting, yaitu algoritma yang berfungsi untuk mengurutkan data berdasarkan urutan tertentu. Proses pengurutan data dapat mempermudah pencarian, pengelompokan, dan pengolahan data dalam berbagai aplikasi.

Pada praktikum ini digunakan beberapa metode sorting, yaitu Bubble Sort, Shell Sort, Quick Sort, dan Merge Sort. Masing-masing algoritma memiliki cara kerja yang berbeda dalam melakukan pengurutan data. Implementasi dilakukan menggunakan bahasa pemrograman Java, sedangkan Bubble Sort juga divisualisasikan menggunakan Java Swing agar proses pengurutan dapat diamati langkah demi langkah.

Melalui praktikum ini diharapkan dapat memahami konsep, cara kerja, serta perbedaan implementasi algoritma Bubble Sort, Shell Sort, Quick Sort, dan Merge Sort dalam proses pengurutan data.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum ini antara lain sebagai berikut:

1. Memahami konsep dan cara kerja algoritma Bubble Sort, Shell Sort, Quick Sort, dan Merge Sort dalam proses pengurutan data.
2. Mengimplementasikan algoritma sorting tersebut menggunakan bahasa pemrograman Java serta memvisualisasikan proses Bubble Sort dengan Java Swing.

1.3 Manfaat

Manfaat dari praktikum ini antara lain sebagai berikut:

1. Meningkatkan pemahaman mengenai perbedaan proses dan karakteristik algoritma Shell Sort, Quick Sort, dan Merge Sort dalam pengurutan data.
2. Meningkatkan kemampuan dalam mengembangkan program Java dan menerapkan algoritma sorting Bubble Sort pada aplikasi berbasis console maupun GUI.

BAB II PEMBAHASAN

2.1 Dasar Teori

Algoritma sorting merupakan salah satu metode yang digunakan untuk mengurutkan data berdasarkan urutan tertentu, baik secara ascending (menaik) maupun descending (menurun). Proses pengurutan data sangat penting dalam pengolahan informasi karena dapat mempermudah pencarian, pengelompokan, dan pengelolaan data. Dengan data yang telah terurut, proses pengolahan data menjadi lebih efisien dan mudah dipahami.

Shell Sort merupakan algoritma pengurutan yang dikembangkan dari Insertion Sort. Algoritma ini menggunakan jarak tertentu (gap) dalam membandingkan elemen sehingga proses pengurutan dapat dilakukan lebih cepat dibandingkan Insertion Sort pada data yang berukuran besar. Nilai gap akan terus diperkecil hingga menjadi satu dan data berada dalam keadaan terurut.

Quick Sort adalah algoritma sorting yang menggunakan metode divide and conquer. Algoritma ini bekerja dengan memilih sebuah pivot, kemudian membagi data menjadi dua bagian, yaitu elemen yang lebih kecil dari pivot dan elemen yang lebih besar dari pivot. Proses tersebut dilakukan secara berulang hingga seluruh data tersusun secara terurut.

Merge Sort juga menggunakan metode divide and conquer. Algoritma ini bekerja dengan membagi array menjadi beberapa bagian yang lebih kecil, kemudian menggabungkannya kembali dalam keadaan terurut. Merge Sort dikenal memiliki performa yang stabil dan efisien untuk pengolahan data dalam jumlah besar.

Bubble Sort merupakan algoritma pengurutan sederhana yang bekerja dengan membandingkan dua elemen yang berdekatan dan menukarnya apabila urutannya tidak sesuai. Pada praktikum ini, Bubble Sort diimplementasikan dalam bentuk GUI menggunakan Java Swing sehingga proses perbandingan dan pertukaran elemen dapat diamati secara visual langkah demi langkah.

2.2 Penjelasan Kode Program

2.2.1 Class ShellSort_2511531007

```
1 package pekan8_2511531007;
2
3 public class ShellSort_2511531007 {
4     public static void shellSort_1007(int[] A_1007) {
5         int n_1007 = A_1007.length;
6         int gap_1007 = n_1007 / 2;
7         while (gap_1007 > 0) {
8             for (int i_1007 = gap_1007; i_1007 < n_1007; i_1007++) {
9                 int temp_1007 = A_1007[i_1007];
10                int j_1007 = i_1007;
11                while (j_1007 >= gap_1007 && A_1007[j_1007 - gap_1007] > temp_1007) {
12                    A_1007[j_1007] = A_1007[j_1007 - gap_1007];
13                    j_1007 = j_1007 - gap_1007;
14                }
15                A_1007[j_1007] = temp_1007;
16            }
17            gap_1007 = gap_1007 / 2;
18        }
19    }
20
21    public static void main(String[] args) {
22        int [] data_1007 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
23
24        System.out.print("Sebelum: ");
25        printArray_1007(data_1007);
26
27        shellSort_1007(data_1007);
28
29        System.out.print("Sesudah (Shell Sort): ");
30        printArray_1007(data_1007);
31    }
32
33    public static void printArray_1007(int [] arr_1007) {
34        for (int i_1007 : arr_1007)
35            System.out.print(i_1007 + " ");
36        System.out.println();
37    }
38
39 }
```

```
Sebelum: 3 10 4 6 8 9 7 2 1 5
Sesudah (Shell Sort): 1 2 3 4 5 6 7 8 9 10
```

Kode program di atas digunakan untuk mengurutkan data menggunakan algoritma Shell Sort. Algoritma ini merupakan pengembangan dari Insertion Sort yang bekerja dengan membandingkan dan mengurutkan elemen-elemen yang memiliki jarak tertentu (gap). Dengan cara ini, data dapat dipindahkan lebih cepat ke posisi yang mendekati urutan yang benar sebelum dilakukan pengurutan akhir.

Di dalam class ShellSort_2511531007 terdapat method shellSort_1007(int[] A_1007) yang berfungsi untuk melakukan proses pengurutan data. Variabel n_1007 digunakan untuk menyimpan jumlah elemen yang terdapat pada array. Selanjutnya variabel gap_1007 diinisialisasi dengan nilai setengah dari panjang array (n_1007 / 2) yang berfungsi sebagai jarak antar elemen yang akan dibandingkan pada setiap tahap pengurutan.

Proses pengurutan dilakukan menggunakan perulangan while selama nilai gap_1007 masih lebih besar dari 0. Di dalam perulangan tersebut terdapat perulangan for yang dimulai dari indeks sebesar gap_1007 hingga elemen terakhir array. Pada setiap perulangan, nilai elemen yang sedang diproses disimpan ke dalam variabel temp_1007. Variabel ini berfungsi sebagai data sementara yang akan ditempatkan pada posisi yang sesuai.

Selanjutnya variabel `j_1007` digunakan untuk menyimpan indeks elemen yang sedang dibandingkan. Perbandingan dilakukan menggunakan perulangan `while`. Selama indeks masih berada dalam batas array dan nilai elemen pada posisi `j_1007 - gap_1007` lebih besar daripada `temp_1007`, maka elemen tersebut akan digeser sejauh `gap_1007` posisi ke kanan. Setelah ditemukan posisi yang sesuai, nilai `temp_1007` ditempatkan pada indeks `j_1007`. Proses ini mirip dengan Insertion Sort, tetapi dilakukan berdasarkan jarak tertentu yang ditentukan oleh nilai `gap_1007`.

Setelah seluruh elemen selesai diproses pada satu tahap, nilai `gap_1007` dibagi dua menggunakan perintah `gap_1007 = gap_1007 / 2`. Proses pengurutan kemudian diulangi dengan jarak yang semakin kecil hingga akhirnya nilai `gap_1007` menjadi 1.

Pada method `main()`, dibuat sebuah array `data_1007` yang berisi data {3, 10, 4, 6, 8, 9, 7, 2, 1, 5}. Program kemudian menampilkan isi array sebelum diurutkan menggunakan method `printArray_1007()`. Setelah itu method `shellSort_1007()` dipanggil untuk melakukan proses pengurutan data. Setelah pengurutan selesai, program kembali menampilkan isi array sehingga pengguna dapat melihat perubahan data dari kondisi belum terurut menjadi data yang sudah terurut secara ascending (dari nilai terkecil ke terbesar).

2.2.2 Class QuickSort_2511531007

```
1 package pkan0_2511531007;
2
3 public class QuickSort_2511531007 {
4     static void swap_1007(int [] arr_1007, int i_1007, int j_1007) {
5         int temp_1007 = arr_1007[i_1007];
6         arr_1007[i_1007] = arr_1007[j_1007];
7         arr_1007[j_1007] = temp_1007;
8     }
9     // metode tambahan untuk mengatur pivot menggunakan median-of-three
10    static void medianOfThree_1007(int [] arr_1007, int low_1007, int high_1007) {
11        int mid_1007 = low_1007 + (high_1007 - low_1007) / 2;
12
13        //urutkan elemen low, mid, dan high
14        if (arr_1007[low_1007] > arr_1007[mid_1007]) {
15            swap_1007(arr_1007, low_1007, mid_1007);
16        }
17        if (arr_1007[low_1007] > arr_1007[high_1007]) {
18            swap_1007(arr_1007, low_1007, high_1007);
19        }
20        if (arr_1007[mid_1007] > arr_1007[high_1007]) {
21            swap_1007(arr_1007, mid_1007, high_1007);
22        }
23        swap_1007(arr_1007, mid_1007, high_1007);
24    }
25    static int partition_1007(int[] arr_1007, int low_1007, int high_1007) {
26        //pilih nilai medianOfThree sebagai pivot
27        medianOfThree_1007(arr_1007, low_1007, high_1007);
28
29        int pivot_1007 = arr_1007[high_1007]; // karena arr_1007[high_1007] sudah berisi nilai median
30        int i_1007 = (low_1007 - 1);
31
32        for (int j_1007 = low_1007; j_1007 <= high_1007 - 1; j_1007++) {
33            // jika elemen saat ini lebih kecil dari atau sama dengan pivot
34            if (arr_1007[j_1007] < pivot_1007) {
35                // increment indeks elemen yang lebih kecil
36                i_1007++;
37                swap_1007(arr_1007, i_1007, j_1007);
38            }
39        }
40        swap_1007(arr_1007, i_1007 + 1, high_1007);
41        return (i_1007 + 1);
42    }
43 }
```

```

43 static void quickSort_1007(int[] arr_1007, int low_1007, int high_1007) {
44     if (low_1007 < high_1007) {
45         int pi_1007 = partition_1007(arr_1007, low_1007, high_1007);
46         quickSort_1007(arr_1007, low_1007, pi_1007 - 1);
47         quickSort_1007(arr_1007, pi_1007 + 1, high_1007);
48     }
49 }
50
51 public static void printArr_1007(int[] arr_1007) {
52     for (int i_1007 = 0; i_1007 < arr_1007.length; i_1007++) {
53         System.out.print(arr_1007[i_1007] + " ");
54     }
55     System.out.println();
56 }
57 public static void main(String[] args) {
58     int[] arr_1007 = { 10, 7, 8, 9, 1, 5 };
59     int N_1007 = arr_1007.length;
60     System.out.print("Data sebelum diurutkan: ");
61     printArr_1007(arr_1007);
62
63     quickSort_1007(arr_1007, 0, N_1007 - 1);
64
65     System.out.print("Data Terurut quicksort: ");
66     printArr_1007(arr_1007);
67 }
68 }
69
70 }

```

```

Data sebelum diurutkan: 10 7 8 9 1 5
Data Terurut quicksort: 1 5 7 8 9 10

```

Kode program di atas digunakan untuk mengurutkan data menggunakan algoritma Quick Sort. Quick Sort merupakan algoritma pengurutan yang bekerja dengan teknik divide and conquer, yaitu membagi data menjadi beberapa bagian yang lebih kecil berdasarkan sebuah nilai acuan yang disebut pivot. Setelah data dibagi, setiap bagian akan diurutkan kembali secara berulang hingga seluruh elemen tersusun secara berurutan dari nilai terkecil ke terbesar.

Di dalam class QuickSort_2511531007 terdapat method swap_1007() yang berfungsi untuk menukar posisi dua elemen dalam array. Method ini menggunakan variabel sementara temp_1007 untuk menyimpan salah satu nilai agar proses pertukaran data dapat dilakukan dengan benar.

Selain itu terdapat method medianOfThree_1007() yang digunakan untuk menentukan pivot menggunakan teknik median-of-three. Teknik ini memilih nilai tengah dari tiga elemen, yaitu elemen pertama (low_1007), elemen tengah (mid_1007), dan elemen terakhir (high_1007). Ketiga elemen tersebut dibandingkan dan diurutkan terlebih dahulu menggunakan method swap_1007(). Setelah itu nilai median dipindahkan ke posisi terakhir array sehingga dapat digunakan sebagai pivot. Penggunaan metode ini bertujuan untuk mengurangi kemungkinan pemilihan pivot yang kurang optimal dan meningkatkan efisiensi proses pengurutan.

Proses utama pengelompokan data dilakukan pada method partition_1007(). Pada awal method, fungsi medianOfThree_1007() dipanggil untuk menentukan pivot. Nilai pivot kemudian disimpan pada variabel pivot_1007, yaitu elemen yang berada pada indeks high_1007. Selanjutnya variabel i_1007 digunakan untuk menandai posisi elemen yang memiliki nilai lebih kecil dari pivot.

Perulangan for digunakan untuk membandingkan setiap elemen array dengan nilai pivot. Jika suatu elemen memiliki nilai lebih kecil dari pivot, maka indeks `i_1007` akan ditambah satu dan elemen tersebut ditukar posisinya menggunakan method `swap_1007()`. Dengan cara ini, seluruh elemen yang lebih kecil dari pivot akan berada di sebelah kiri, sedangkan elemen yang lebih besar akan berada di sebelah kanan. Setelah proses perbandingan selesai, pivot dipindahkan ke posisi yang sesuai menggunakan method `swap_1007()`. Method `partition_1007()` kemudian mengembalikan indeks pivot yang baru melalui variabel `i_1007 + 1`.

Method `quickSort_1007()` merupakan method rekursif yang digunakan untuk menjalankan algoritma Quick Sort. Jika nilai `low_1007` masih lebih kecil dari `high_1007`, maka method `partition_1007()` dipanggil untuk menentukan posisi pivot. Setelah posisi pivot diperoleh dan disimpan pada variabel `pi_1007`, method `quickSort_1007()` akan dipanggil kembali untuk mengurutkan bagian kiri pivot serta bagian kanan pivot. Proses ini berlangsung secara berulang hingga seluruh bagian array selesai diurutkan.

Pada method `main()`, dibuat sebuah array `arr_1007` yang berisi data {10, 7, 8, 9, 1, 5}. Program kemudian menampilkan isi array sebelum diurutkan menggunakan method `printArr_1007()`. Setelah itu method `quickSort_1007()` dipanggil dengan parameter indeks awal 0 dan indeks akhir `N_1007 - 1` untuk melakukan proses pengurutan data. Setelah seluruh proses selesai, program kembali menampilkan isi array sehingga pengguna dapat melihat hasil pengurutan dari data yang belum terurut menjadi data yang sudah terurut secara ascending (dari nilai terkecil ke terbesar).

2.2.3 Class MergeSort_2511531007

```
1 package pekan8_2511531007;
2
3 public class MergeSort_2511531007 {
4     void merge(int arr[], int l_1007, int m_1007, int r_1007) {
5         // Find sizes of two subarrays to be merged
6         int n1_1007 = m_1007 - l_1007 + 1;
7         int n2_1007 = r_1007 - m_1007;
8         /* Create temp arrays */
9         int L_1007[] = new int[n1_1007];
10        int R_1007[] = new int[n2_1007];
11        /* Copy data to temp arrays */
12        for (int i_1007 = 0; i_1007 < n1_1007; ++i_1007)
13            L_1007[i_1007] = arr[l_1007 + i_1007];
14        for (int j_1007 = 0; j_1007 < n2_1007; ++j_1007)
15            R_1007[j_1007] = arr[m_1007 + 1 + j_1007];
16        int i_1007 = 0, j_1007 = 0;
17        // Initial index of merged subarray array
18        int k_1007 = l_1007;
19        while (i_1007 < n1_1007 && j_1007 < n2_1007) {
20            if (L_1007[i_1007] <= R_1007[j_1007]) {
21                arr[k_1007] = L_1007[i_1007];
22                i_1007++;
23            } else {
24                arr[k_1007] = R_1007[j_1007];
25                j_1007++;
26            }
27            k_1007++;
28        }
29        /* Copy remaining elements of L[] if any */
30        while (i_1007 < n1_1007) {
31            arr[k_1007] = L_1007[i_1007];
32            i_1007++;
33            k_1007++;
34        }
35        /* Copy remaining elements of R[] if any */
36        while (j_1007 < n2_1007) {
37            arr[k_1007] = R_1007[j_1007];
38            j_1007++;
39            k_1007++;
40        }
41    }
42}
```

```
42 void sort(int arr_1007[], int l_1007, int r_1007) {
43     if (l_1007 < r_1007) {
44         // Find the middle point
45         int m_1007 = (l_1007 + r_1007) / 2;
46         // Sort first and second halves
47         sort(arr_1007, l_1007, m_1007);
48         sort(arr_1007, m_1007 + 1, r_1007);
49         // Merge the sorted halves
50         merge(arr_1007, l_1007, m_1007, r_1007);
51     }
52 }
53
54 /* A utility function to print array of size n */
55 static void printArray(int arr_1007[]) {
56     int n_1007 = arr_1007.length;
57     for (int i_1007 = 0; i_1007 < n_1007; ++i_1007)
58         System.out.print(arr_1007[i_1007] + " ");
59     System.out.println();
60 }
61
62 public static void main(String args[]) {
63     int arr_1007[] = { 12, 11, 13, 5, 6, 7 };
64     System.out.println("Sebelum terurut");
65     printArray(arr_1007);
66     MergeSort_2511531007 ob_1007 = new MergeSort_2511531007();
67     ob_1007.sort(arr_1007, 0, arr_1007.length - 1);
68     System.out.println("\nSesudah Terurut menggunakan merge Sort");
69     printArray(arr_1007);
70 }
71 }
```

```
Sebelum terurut
12 11 13 5 6 7

Sesudah Terurut menggunakan merge Sort
5 6 7 11 12 13
```

Kode program di atas digunakan untuk mengurutkan data menggunakan algoritma Merge Sort. Algoritma ini bekerja dengan metode divide and conquer, yaitu membagi array menjadi beberapa bagian yang lebih kecil hingga setiap bagian hanya terdiri dari satu elemen. Setelah itu, bagian-bagian tersebut digabungkan kembali secara bertahap dalam keadaan terurut hingga menghasilkan satu array yang seluruh elemennya tersusun dari nilai terkecil ke terbesar.

Di dalam class MergeSort_2511531007 terdapat method merge() yang berfungsi untuk menggabungkan dua subarray yang sudah terurut menjadi satu array yang terurut. Parameter l_1007, m_1007, dan r_1007 digunakan untuk menentukan batas kiri, titik tengah, dan batas kanan dari bagian array yang akan digabungkan. Variabel n1_1007 dan n2_1007 digunakan untuk menyimpan jumlah elemen pada subarray kiri dan subarray kanan.

Selanjutnya dibuat dua array sementara, yaitu L_1007[] dan R_1007[], yang digunakan untuk menyimpan data dari subarray kiri dan kanan. Proses penyalinan data dilakukan menggunakan perulangan for sehingga elemen-elemen yang akan dibandingkan tidak langsung memengaruhi isi array utama.

Setelah data berhasil disalin, proses penggabungan dilakukan menggunakan perulangan while. Variabel i_1007 digunakan sebagai indeks untuk subarray kiri, j_1007 sebagai indeks untuk subarray kanan, dan k_1007 sebagai indeks untuk array utama. Pada setiap perulangan, elemen dari kedua subarray dibandingkan. Jika nilai pada L_1007[i_1007] lebih kecil atau sama dengan R_1007[j_1007], maka nilai tersebut ditempatkan ke dalam array utama. Sebaliknya, jika nilai pada subarray kanan lebih kecil, maka elemen dari R_1007 yang dimasukkan ke array utama. Proses ini terus dilakukan hingga salah satu subarray habis diproses.

Setelah salah satu subarray selesai dibandingkan, masih mungkin terdapat elemen yang belum dipindahkan pada subarray lainnya. Oleh karena itu digunakan dua perulangan while tambahan untuk menyalin sisa elemen dari L_1007 atau R_1007 ke array utama. Dengan cara ini seluruh elemen dapat tergabung kembali dalam keadaan terurut.

Method sort() merupakan method rekursif yang digunakan untuk membagi array menjadi bagian-bagian yang lebih kecil. Jika nilai l_1007 masih lebih kecil dari r_1007, maka titik tengah array dihitung dan disimpan pada variabel m_1007. Selanjutnya method sort() dipanggil kembali untuk mengurutkan bagian kiri dan bagian kanan array secara terpisah. Setelah kedua bagian selesai diurutkan, method merge() dipanggil untuk menggabungkan keduanya menjadi satu bagian yang sudah terurut.

Selain itu terdapat method printArray() yang berfungsi untuk menampilkan seluruh elemen array ke layar menggunakan perulangan for. Method ini digunakan untuk memperlihatkan kondisi data sebelum dan sesudah proses pengurutan.

Pada method main(), dibuat sebuah array arr_1007 yang berisi data {12, 11, 13, 5, 6, 7}. Program kemudian menampilkan isi array sebelum diurutkan menggunakan method printArray(). Setelah itu dibuat objek ob_1007 dari class MergeSort_2511531007 dan method sort() dipanggil untuk melakukan proses pengurutan data. Setelah seluruh proses selesai, program kembali menampilkan isi array sehingga pengguna dapat melihat perubahan data dari kondisi belum terurut menjadi data yang sudah terurut secara ascending (dari nilai terkecil ke terbesar).

2.2.4 Class BubbleSortGUI_2511531007

```

1 package pekan8_2511531007;
2
3 import java.awt.*;
4 import javax.swing.*;
5
6 public class BubbleSortGUI_2511531007 extends JFrame {
7     private static final long serialVersionUID = 1L;
8     private int[] array_1007;
9     private JLabel [] labelArray_1007;
10    private JButton stepButton_1007, resetButton_1007, setButton_1007;
11    private JTextField inputField_1007;
12    private JPanel panelArray_1007;
13    private JTextArea stepArea_1007;
14
15    private int i_1007 = 1, j_1007;
16    private boolean sorting_1007 = false;
17    private int stepCount_1007 = 1;
18
19    public BubbleSortGUI_2511531007() {
20        setTitle("Bubble Sort Langkah per Langkah");
21        setSize(750, 400);
22        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
23        setLocationRelativeTo(null);
24        setLayout(new BorderLayout());
25
26        // Panel input
27        JPanel inputPanel_1007 = new JPanel(new FlowLayout());
28        inputField_1007 = new JTextField(30);
29        setButton_1007 = new JButton("Set Array");
30        inputPanel_1007.add(new JLabel("Masukkan angka (pisahkan dengan koma): "));
31        inputPanel_1007.add(inputField_1007);
32        inputPanel_1007.add(setButton_1007);
33
34        // Panel array visual
35        panelArray_1007 = new JPanel();
36        panelArray_1007.setLayout(new FlowLayout());
37
38        // Panel kontrol
39        JPanel controlPanel_1007 = new JPanel();
40        stepButton_1007 = new JButton("Langkah Selanjutnya");
41        resetButton_1007 = new JButton("Reset");
42        stepButton_1007.setEnabled(false);
43        controlPanel_1007.add(stepButton_1007);
44        controlPanel_1007.add(resetButton_1007);
45
46        // Area teks untuk log langkah-langkah
47        stepArea_1007 = new JTextArea(8, 60);
48        stepArea_1007.setEditable(false);
49        stepArea_1007.setFont(new Font("Monospaced", Font.PLAIN, 14));
50        JScrollPane scrollPane_1007 = new JScrollPane(stepArea_1007);
51
52        // Tambahkan panel ke frame
53        add(inputPanel_1007, BorderLayout.NORTH);
54        add(panelArray_1007, BorderLayout.CENTER);
55        add(controlPanel_1007, BorderLayout.EAST);
56        add(scrollPane_1007, BorderLayout.SOUTH);
57
58        // Event Set Array
59        setButton_1007.addActionListener(e -> setArrayFromInput());
60
61        // Event Langkah Selanjutnya
62        stepButton_1007.addActionListener(e -> performStep_1007());
63
64        // Event Reset
65        resetButton_1007.addActionListener(e -> reset());
66    }
67
68    private void setArrayFromInput() {
69
70    }
71
72    private void performStep_1007() {
73
74    }
75
76    private void reset() {
77
78    }
79
80    private void printArray() {
81
82    }
83
84    private void sort() {
85
86    }
87
88    private void swap(int i, int j) {
89
90    }
91
92    private void bubbleSort() {
93
94    }
95
96    private void step() {
97
98    }
99
100   private void resetStepCount() {
101
102   }
103
104   private void enableStepButton() {
105
106   }
107
108   private void disableStepButton() {
109
110   }
111
112   private void updateLabelArray() {
113
114   }
115
116   private void updateStepArea() {
117
118   }
119
120   private void updateSortingStatus() {
121
122   }
123
124   private void updateStepCount() {
125
126   }
127
128   private void updateArray() {
129
130   }
131
132   private void updateLabelArray_1007() {
133
134   }
135
136   private void updateStepArea_1007() {
137
138   }
139
140   private void updateSortingStatus_1007() {
141
142   }
143
144   private void updateStepCount_1007() {
145
146   }
147
148   private void updateArray_1007() {
149
150   }
151
152   private void updateLabelArray_1007_1() {
153
154   }
155
156   private void updateStepArea_1007_1() {
157
158   }
159
160   private void updateSortingStatus_1007_1() {
161
162   }
163
164   private void updateStepCount_1007_1() {
165
166   }
167
168   private void updateArray_1007_1() {
169
170   }
171
172   private void updateLabelArray_1007_2() {
173
174   }
175
176   private void updateStepArea_1007_2() {
177
178   }
179
180   private void updateSortingStatus_1007_2() {
181
182   }
183
184   private void updateStepCount_1007_2() {
185
186   }
187
188   private void updateArray_1007_2() {
189
190   }
191
192   private void updateLabelArray_1007_3() {
193
194   }
195
196   private void updateStepArea_1007_3() {
197
198   }
199
200   private void updateSortingStatus_1007_3() {
201
202   }
203
204   private void updateStepCount_1007_3() {
205
206   }
207
208   private void updateArray_1007_3() {
209
210   }
211
212   private void updateLabelArray_1007_4() {
213
214   }
215
216   private void updateStepArea_1007_4() {
217
218   }
219
220   private void updateSortingStatus_1007_4() {
221
222   }
223
224   private void updateStepCount_1007_4() {
225
226   }
227
228   private void updateArray_1007_4() {
229
230   }
231
232   private void updateLabelArray_1007_5() {
233
234   }
235
236   private void updateStepArea_1007_5() {
237
238   }
239
240   private void updateSortingStatus_1007_5() {
241
242   }
243
244   private void updateStepCount_1007_5() {
245
246   }
247
248   private void updateArray_1007_5() {
249
250   }
251
252   private void updateLabelArray_1007_6() {
253
254   }
255
256   private void updateStepArea_1007_6() {
257
258   }
259
260   private void updateSortingStatus_1007_6() {
261
262   }
263
264   private void updateStepCount_1007_6() {
265
266   }
267
268   private void updateArray_1007_6() {
269
270   }
271
272   private void updateLabelArray_1007_7() {
273
274   }
275
276   private void updateStepArea_1007_7() {
277
278   }
279
280   private void updateSortingStatus_1007_7() {
281
282   }
283
284   private void updateStepCount_1007_7() {
285
286   }
287
288   private void updateArray_1007_7() {
289
290   }
291
292   private void updateLabelArray_1007_8() {
293
294   }
295
296   private void updateStepArea_1007_8() {
297
298   }
299
300   private void updateSortingStatus_1007_8() {
301
302   }
303
304   private void updateStepCount_1007_8() {
305
306   }
307
308   private void updateArray_1007_8() {
309
310   }
311
312   private void updateLabelArray_1007_9() {
313
314   }
315
316   private void updateStepArea_1007_9() {
317
318   }
319
320   private void updateSortingStatus_1007_9() {
321
322   }
323
324   private void updateStepCount_1007_9() {
325
326   }
327
328   private void updateArray_1007_9() {
329
330   }
331
332   private void updateLabelArray_1007_10() {
333
334   }
335
336   private void updateStepArea_1007_10() {
337
338   }
339
340   private void updateSortingStatus_1007_10() {
341
342   }
343
344   private void updateStepCount_1007_10() {
345
346   }
347
348   private void updateArray_1007_10() {
349
350   }
351
352   private void updateLabelArray_1007_11() {
353
354   }
355
356   private void updateStepArea_1007_11() {
357
358   }
359
360   private void updateSortingStatus_1007_11() {
361
362   }
363
364   private void updateStepCount_1007_11() {
365
366   }
367
368   private void updateArray_1007_11() {
369
370   }
371
372   private void updateLabelArray_1007_12() {
373
374   }
375
376   private void updateStepArea_1007_12() {
377
378   }
379
380   private void updateSortingStatus_1007_12() {
381
382   }
383
384   private void updateStepCount_1007_12() {
385
386   }
387
388   private void updateArray_1007_12() {
389
390   }
391
392   private void updateLabelArray_1007_13() {
393
394   }
395
396   private void updateStepArea_1007_13() {
397
398   }
399
400   private void updateSortingStatus_1007_13() {
401
402   }
403
404   private void updateStepCount_1007_13() {
405
406   }
407
408   private void updateArray_1007_13() {
409
410   }
411
412   private void updateLabelArray_1007_14() {
413
414   }
415
416   private void updateStepArea_1007_14() {
417
418   }
419
420   private void updateSortingStatus_1007_14() {
421
422   }
423
424   private void updateStepCount_1007_14() {
425
426   }
427
428   private void updateArray_1007_14() {
429
430   }
431
432   private void updateLabelArray_1007_15() {
433
434   }
435
436   private void updateStepArea_1007_15() {
437
438   }
439
440   private void updateSortingStatus_1007_15() {
441
442   }
443
444   private void updateStepCount_1007_15() {
445
446   }
447
448   private void updateArray_1007_15() {
449
450   }
451
452   private void updateLabelArray_1007_16() {
453
454   }
455
456   private void updateStepArea_1007_16() {
457
458   }
459
460   private void updateSortingStatus_1007_16() {
461
462   }
463
464   private void updateStepCount_1007_16() {
465
466   }
467
468   private void updateArray_1007_16() {
469
470   }
471
472   private void updateLabelArray_1007_17() {
473
474   }
475
476   private void updateStepArea_1007_17() {
477
478   }
479
480   private void updateSortingStatus_1007_17() {
481
482   }
483
484   private void updateStepCount_1007_17() {
485
486   }
487
488   private void updateArray_1007_17() {
489
490   }
491
492   private void updateLabelArray_1007_18() {
493
494   }
495
496   private void updateStepArea_1007_18() {
497
498   }
499
500   private void updateSortingStatus_1007_18() {
501
502   }
503
504   private void updateStepCount_1007_18() {
505
506   }
507
508   private void updateArray_1007_18() {
509
510   }
511
512   private void updateLabelArray_1007_19() {
513
514   }
515
516   private void updateStepArea_1007_19() {
517
518   }
519
520   private void updateSortingStatus_1007_19() {
521
522   }
523
524   private void updateStepCount_1007_19() {
525
526   }
527
528   private void updateArray_1007_19() {
529
530   }
531
532   private void updateLabelArray_1007_20() {
533
534   }
535
536   private void updateStepArea_1007_20() {
537
538   }
539
540   private void updateSortingStatus_1007_20() {
541
542   }
543
544   private void updateStepCount_1007_20() {
545
546   }
547
548   private void updateArray_1007_20() {
549
550   }
551
552   private void updateLabelArray_1007_21() {
553
554   }
555
556   private void updateStepArea_1007_21() {
557
558   }
559
560   private void updateSortingStatus_1007_21() {
561
562   }
563
564   private void updateStepCount_1007_21() {
565
566   }
567
568   private void updateArray_1007_21() {
569
570   }
571
572   private void updateLabelArray_1007_22() {
573
574   }
575
576   private void updateStepArea_1007_22() {
577
578   }
579
580   private void updateSortingStatus_1007_22() {
581
582   }
583
584   private void updateStepCount_1007_22() {
585
586   }
587
588   private void updateArray_1007_22() {
589
590   }
589

```

```

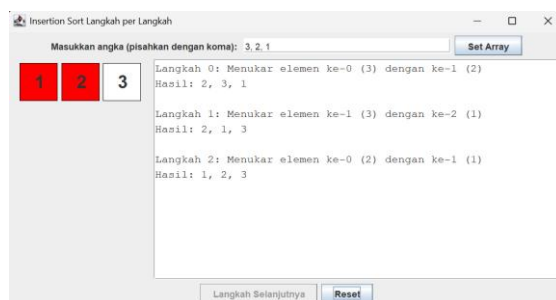
105 private void performStep_1007() {
106
107     if (!sorting_1007 || i_1007 > array_1007.length - 1) {
108         sorting_1007 = false;
109         stepButton_1007.setEnabled(false);
110         JOptionPane.showMessageDialog(this, "Sorting selesai!");
111         return;
112     }
113     resetHighlights_1007();
114
115     StringBuilder stepLog_1007 = new StringBuilder();
116     labelArray_1007[j_1007].setBackground(Color.CYAN);
117     labelArray_1007[j_1007 + 1].setBackground(Color.CYAN);
118
119     if (array_1007[j_1007] > array_1007[j_1007 + 1]) {
120         int temp_1007 = array_1007[j_1007];
121         array_1007[j_1007] = array_1007[j_1007 + 1];
122         array_1007[j_1007 + 1] = temp_1007;
123         labelArray_1007[j_1007].setBackground(Color.RED);
124         labelArray_1007[j_1007 + 1].setBackground(Color.RED);
125
126         stepLog_1007.append("Langkah ").append(stepCount_1007)
127             .append(": Menukar elemen ke-")
128             .append(j_1007).append(" ")
129             .append(array_1007[j_1007 + 1]).append(" dengan ke-")
130             .append(j_1007 + 1).append(" ").append("\n");
131         .append(array_1007[j_1007]).append("\n");
132     } else {
133         stepLog_1007.append("Langkah ").append(stepCount_1007)
134             .append(": Tidak ada pertukaran antara ke-")
135             .append(j_1007).append(" dan ke-")
136             .append(j_1007 + 1).append("\n");
137     }
138
139     stepLog_1007.append("Hasil: ")
140         .append(arrayToString(array_1007))
141         .append("\n\n");
142
143     stepArea_1007.append(stepLog_1007.toString());
144 }
145
146
147
148
149
150
151
152
153
154
155
156
157

```

```

131 updateLabels();
132 j_1007++;
133
134 if (j_1007 >= array_1007.length - 1 - i_1007) {
135     j_1007 = 0;
136     i_1007++;
137     if (i_1007 >= array_1007.length - 1) {
138         sorting_1007 = false;
139         stepButton_1007.setEnabled(false);
140         JOptionPane.showMessageDialog(this, "Sorting selesai!");
141     }
142     stepCount_1007++;
143 }
144
145 private void resetHighlights_1007() {
146     for (JLabel label_1007 : labelArray_1007) {
147         label_1007.setBackground(Color.WHITE);
148     }
149 }
150
151 private void updateLabels() {
152     for (int k_1007 = 0; k_1007 < array_1007.length; k_1007++) {
153         labelArray_1007[k_1007].setText(String.valueOf(array_1007[k_1007]));
154     }
155 }
156
157 private void reset() {
158     inputField_1007.setText("");
159     panelArray_1007.removeAll();
160     panelArray_1007.revalidate();
161     panelArray_1007.repaint();
162     stepArea_1007.setText("");
163     stepButton_1007.setEnabled(false);
164     sorting_1007 = false;
165     i_1007 = 0;
166     j_1007 = 0;
167     stepCount_1007 = 1;
168 }
169
170 private String arrayToString(int[] arr) {
171     StringBuilder sb = new StringBuilder();
172     for (int k = 0; k < arr.length; k++) {
173         sb.append(arr[k]);
174         if (k < arr.length - 1) sb.append(", ");
175     }
176     return sb.toString();
177 }
178
179 public static void main(String[] args) {
180     SwingUtilities.invokeLater(new Runnable() {
181         @Override
182         public void run() {
183             new BubbleSortGUI_2511531007().setVisible(true);
184         }
185     });
186 }
187 }

```



Kode program di atas merupakan aplikasi Bubble Sort berbasis GUI (*Graphical User Interface*) yang dibuat menggunakan library Java Swing. Program ini digunakan

untuk melakukan proses pengurutan data menggunakan algoritma Bubble Sort secara langkah demi langkah. Selain menampilkan hasil pengurutan, program juga memberikan visualisasi perbandingan dan pertukaran elemen sehingga pengguna dapat memahami cara kerja algoritma Bubble Sort dengan lebih mudah.

Di dalam class `BubbleSortGUI_2511531007` terdapat beberapa komponen antarmuka seperti `TextField` untuk memasukkan data, `Button` untuk menjalankan proses, `Label` untuk menampilkan elemen array secara visual, serta `TextArea` yang digunakan untuk menampilkan catatan atau log setiap langkah pengurutan. Variabel `array_1007` digunakan untuk menyimpan data yang akan diurutkan, sedangkan `labelArray_1007` digunakan untuk menampilkan setiap elemen array dalam bentuk kotak-kotak pada layar.

Pada konstruktor `BubbleSortGUI_2511531007()`, dilakukan proses pembuatan dan pengaturan tampilan GUI. Program membuat panel input untuk memasukkan angka, panel visualisasi array untuk menampilkan data, panel kontrol yang berisi tombol Set Array, Langkah Selanjutnya, dan Reset, serta area teks untuk menampilkan penjelasan proses sorting. Selain itu, event listener juga ditambahkan pada setiap tombol agar dapat menjalankan fungsi yang sesuai ketika ditekan.

Method `setArrayFromInput()` berfungsi untuk membaca data yang dimasukkan oleh pengguna melalui `inputField_1007`. Data yang dipisahkan dengan tanda koma akan dipecah menggunakan method `split()`, kemudian dikonversi menjadi bilangan bulat dan disimpan ke dalam array `array_1007`. Jika pengguna memasukkan data yang bukan angka, program akan menampilkan pesan kesalahan menggunakan `JOptionPane`. Setelah data berhasil dimasukkan, program membuat sejumlah `JLabel` sesuai banyaknya elemen array dan menampilkannya pada panel visualisasi.

Method `performStep_1007()` merupakan bagian utama dari proses Bubble Sort. Method ini menjalankan satu langkah pengurutan setiap kali tombol Langkah Selanjutnya ditekan. Program terlebih dahulu memberi warna CYAN pada dua elemen yang sedang dibandingkan. Jika nilai elemen pertama lebih besar daripada elemen kedua, maka dilakukan pertukaran posisi menggunakan variabel sementara `temp_1007`. Elemen yang ditukar kemudian diberi warna RED sebagai penanda bahwa telah terjadi pertukaran data.

Setelah proses perbandingan selesai, program mencatat informasi langkah tersebut ke dalam `stepArea_1007`. Informasi yang ditampilkan meliputi nomor langkah, elemen

yang dibandingkan, apakah terjadi pertukaran atau tidak, serta kondisi array setelah proses dilakukan. Dengan cara ini pengguna dapat melihat setiap perubahan yang terjadi selama proses Bubble Sort berlangsung.

Variabel `i_1007` digunakan untuk menghitung jumlah putaran (pass) Bubble Sort, sedangkan `j_1007` digunakan untuk menunjukkan posisi elemen yang sedang dibandingkan. Setelah satu putaran selesai, nilai `j_1007` dikembalikan ke nol dan `i_1007` ditambah satu. Proses ini terus berlangsung hingga seluruh data berhasil diurutkan. Ketika pengurutan selesai, tombol Langkah Selanjutnya dinonaktifkan dan program menampilkan pesan bahwa proses sorting telah selesai.

Method `resetHighlights_1007()` digunakan untuk mengembalikan warna seluruh label menjadi putih sebelum langkah berikutnya dijalankan. Method `updateLabels()` berfungsi untuk memperbarui tampilan nilai pada setiap label setelah terjadi pertukaran data. Sementara itu, method `reset()` digunakan untuk mengembalikan seluruh kondisi program ke keadaan awal, seperti menghapus input, membersihkan panel visualisasi, mengosongkan area log, serta mengatur ulang nilai variabel yang digunakan selama proses pengurutan.

Method `arrayToString()` berfungsi untuk mengubah isi array menjadi bentuk teks sehingga dapat ditampilkan pada area log. Method ini menggabungkan seluruh elemen array menjadi satu string yang dipisahkan oleh tanda koma.

Pada method `main()`, program dijalankan menggunakan `SwingUtilities.invokeLater()` untuk memastikan seluruh komponen GUI dibuat dan dijalankan pada *Event Dispatch Thread* (EDT). Setelah itu objek `BubbleSortGUI_2511531007` dibuat dan ditampilkan menggunakan method `setVisible(true)` sehingga pengguna dapat langsung berinteraksi dengan aplikasi.

Program ini tidak hanya menampilkan hasil akhir pengurutan, tetapi juga memperlihatkan setiap proses perbandingan dan pertukaran data yang terjadi pada algoritma Bubble Sort. Pendekatan seperti ini sangat membantu dalam proses pembelajaran karena pengguna dapat melihat secara langsung bagaimana elemen-elemen bergerak hingga akhirnya tersusun secara ascending (dari nilai terkecil ke terbesar).

2.3 Tugas Praktikum

2.3.1 Class Lagu_2511531007

```
1 package tugasPraktikum_ekan0_2511531007;
2
3 public class Lagu_2511531007 {
4     private String judul_1007;
5     private String penyanyi_1007;
6     private int durasi_1007;
7
8     // Konstruktor
9     Lagu_2511531007(String judul_1007, String penyanyi_1007, int durasi_1007) {
10         this.judul_1007 = judul_1007;
11         this.penyanyi_1007 = penyanyi_1007;
12         this.durasi_1007 = durasi_1007;
13     }
14
15     // Getter
16     public String getJudul_1007() {return judul_1007;}
17     public String getPenyanyi_1007() {return penyanyi_1007;}
18     public int getDurasi_1007() {return durasi_1007;}
19
20     // Setter
21     public void setJudul_1007(String judul_1007) {this.judul_1007 = judul_1007;}
22     public void setPenyanyi_1007(String penyanyi_1007) {this.penyanyi_1007 = penyanyi_1007;}
23     public void setDurasi_1007(int durasi_1007) {this.durasi_1007 = durasi_1007;}
24
25     public String toString_1007() {
26         return judul_1007 + " - " + penyanyi_1007 + "(" + durasi_1007 + " detik)";
27     }
28 }
```

Kode program di atas digunakan untuk membuat sebuah class `Lagu_2511531007` yang berfungsi sebagai representasi objek lagu dalam program playlist. Class ini digunakan untuk menyimpan informasi mengenai lagu, seperti judul lagu, nama penyanyi, dan durasi lagu.

Di dalam class `Lagu_2511531007` terdapat tiga atribut private, yaitu `judul_1007`, `penyanyi_1007`, dan `durasi_1007`. Atribut `judul_1007` bertipe `String` digunakan untuk menyimpan judul lagu, atribut `penyanyi_1007` bertipe `String` digunakan untuk menyimpan nama penyanyi, sedangkan atribut `durasi_1007` bertipe `int` digunakan untuk menyimpan lama durasi lagu dalam satuan detik. Penggunaan access modifier `private` bertujuan untuk menjaga keamanan data agar tidak dapat diakses secara langsung dari luar class.

Class ini memiliki sebuah konstruktor bernama `Lagu_2511531007()` yang digunakan untuk menginisialisasi nilai atribut saat objek dibuat. Konstruktor menerima tiga parameter, yaitu judul lagu, nama penyanyi, dan durasi lagu. Nilai parameter tersebut kemudian disimpan ke dalam atribut class menggunakan kata kunci `this` untuk membedakan antara atribut dan parameter yang memiliki nama yang sama.

Selain konstruktor, class ini juga memiliki beberapa method getter, yaitu `getJudul_1007()`, `getPenyanyi_1007()`, dan `getDurasi_1007()`. Method getter berfungsi untuk mengambil atau membaca nilai dari atribut yang bersifat `private` sehingga data tetap dapat diakses dengan aman dari luar class.

Class ini juga menyediakan method setter, yaitu `setJudul_1007()`, `setPenyanyi_1007()`, dan `setDurasi_1007()`. Method setter digunakan untuk mengubah nilai atribut setelah objek berhasil dibuat. Dengan adanya getter dan setter, konsep

enkapsulasi (encapsulation) dalam pemrograman berorientasi objek dapat diterapkan dengan baik.

Pada bagian akhir terdapat method `toString_1007()` yang berfungsi untuk menggabungkan informasi lagu menjadi sebuah teks yang mudah dibaca. Method ini mengembalikan nilai berupa judul lagu, nama penyanyi, dan durasi lagu dalam format: Judul Lagu - Nama Penyanyi (Durasi detik).

2.3.2 Class Sorting_2511531007

```
1 package tugasPraktikum_pekan0_2511531007;
2
3 public class Sorting_2511531007 {
4     Lagu_2511531007[] dataLagu_1007 = new Lagu_2511531007[20];
5     int jumlah_1007 = 0;
6
7     public void inputData_1007() {
8         dataLagu_1007[0] = new Lagu_2511531007("Feel It", "David", 137);
9         dataLagu_1007[1] = new Lagu_2511531007("In My Room", "Chance Peña", 157);
10        dataLagu_1007[2] = new Lagu_2511531007("Unforgettable", "Niyora", 122);
11        dataLagu_1007[3] = new Lagu_2511531007("Nobody's Better", "Soul ft. Fetty Wap", 209);
12        dataLagu_1007[4] = new Lagu_2511531007("Her", "JIVE ft. ZVC", 144);
13        dataLagu_1007[5] = new Lagu_2511531007("On Bended Knee", "Boyz II Men", 329);
14        dataLagu_1007[6] = new Lagu_2511531007("Love", "Keyshia Cole", 255);
15        jumlah_1007 = 7;
16    }
17
18    public void tampilData_1007() {
19        for (int i_1007 = 0; i_1007 < jumlah_1007; ++i_1007) {
20            System.out.println((i_1007 + 1) + " : " + dataLagu_1007[i_1007].toString_1007());
21        }
22    }
23
24    // swap dua elemen array lagu
25    static void swap_1007(Lagu_2511531007[] arr_1007, int i_1007, int j_1007) {
26        Lagu_2511531007 temp_1007 = arr_1007[i_1007];
27        arr_1007[i_1007] = arr_1007[j_1007];
28        arr_1007[j_1007] = temp_1007;
29    }
30
31    // metode tambahan untuk mengatur pivot menggunakan median-of-three
32    static void medianOfThree_1007(Lagu_2511531007[] arr_1007, int low_1007, int high_1007) {
33        int mid_1007 = low_1007 + (high_1007 - low_1007) / 2;
34
35        // urutkan berdasarkan durasi: low, mid, high
36        if (arr_1007[low_1007].getDurasi_1007() > arr_1007[mid_1007].getDurasi_1007()) {
37            swap_1007(arr_1007, low_1007, mid_1007);
38        }
39        if (arr_1007[low_1007].getDurasi_1007() > arr_1007[high_1007].getDurasi_1007()) {
40            swap_1007(arr_1007, low_1007, high_1007);
41        }
42        if (arr_1007[mid_1007].getDurasi_1007() > arr_1007[high_1007].getDurasi_1007()) {
43            swap_1007(arr_1007, mid_1007, high_1007);
44        }
45        // Pindahkan median ke posisi high sebagai pivot
46        swap_1007(arr_1007, mid_1007, high_1007);
47    }
48
49    static int partition_1007(Lagu_2511531007[] arr_1007, int low_1007, int high_1007) {
50        // panggil fungsi medianOfThree sebelum menentukan pivot
51        medianOfThree_1007(arr_1007, low_1007, high_1007);
52
53        int pivot_1007 = arr_1007[high_1007].getDurasi_1007(); // sekarang arr_1007[high_1007] sudah berisi nilai median dari durasi
54        int i_1007 = (low_1007 - 1);
55
56        for (int j_1007 = low_1007; j_1007 <= high_1007 - 1; j_1007++) {
57            // membandingkan durasi
58            if (arr_1007[j_1007].getDurasi_1007() < pivot_1007) {
59                // increment indeks elemen yang lebih kecil
60                i_1007++;
61                swap_1007(arr_1007, i_1007, j_1007);
62            }
63        }
64        swap_1007(arr_1007, i_1007 + 1, high_1007);
65        return (i_1007 + 1);
66    }
67
68    // method utama quick sort
69    static void quickSort_1007(Lagu_2511531007[] arr_1007, int low_1007, int high_1007) {
70        if (low_1007 < high_1007) {
71            int pi_1007 = partition_1007(arr_1007, low_1007, high_1007);
72            quickSort_1007(arr_1007, low_1007, pi_1007 - 1);
73            quickSort_1007(arr_1007, pi_1007 + 1, high_1007);
74        }
75    }
76
77    // wrapper method untuk memudahkan panggilan dari luar
78    public void quickSort_1007() {
79        if (jumlah_1007 > 1) {
80            quickSort_1007(dataLagu_1007, 0, jumlah_1007 - 1);
81        }
82    }
83
84    public static void main(String[] args) {
85        Sorting_2511531007 playlist_1007 = new Sorting_2511531007();
86
87        System.out.println("=== Sorting Playlist RDN: 2511531007 ===");
88        System.out.println("Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 2");
89        playlist_1007.inputData_1007();
90
91        System.out.println("Data Sebelum Sorting:");
92        playlist_1007.tampilData_1007();
93
94        // Eksekusi Quick Sort
95        playlist_1007.quickSort_1007();
96
97        System.out.println("Data Setelah Quick Sort (Durasi Asc):");
98        playlist_1007.tampilData_1007();
99    }
100 }
```

```

=== Sorting Playlist NIM: 2511531007 ===
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 2

Data Sebelum Sorting:
1.Feel It - D4vd (137 detik)
2.In My Room - Chance Peña (157 detik)
3.Unforgettable - Miyuze (122 detik)
4.Nobody's Better - Suzi ft. Fetty Wap (209 detik)
5.Her - JVKE ft. ZVC (144 detik)
6.On Bended Knee - Boyz II Men (329 detik)
7.Love - Keyshia Cole (255 detik)

Data Setelah Quick Sort (Durasi Asc):
1.Unforgettable - Miyuze (122 detik)
2.Feel It - D4vd (137 detik)
3.Her - JVKE ft. ZVC (144 detik)
4.In My Room - Chance Peña (157 detik)
5.Nobody's Better - Suzi ft. Fetty Wap (209 detik)
6.Love - Keyshia Cole (255 detik)
7.On Bended Knee - Boyz II Men (329 detik)

```

Saya memilih Quick Sort karena menurut saya algoritma ini cukup menarik untuk dipelajari dan cara kerjanya juga berbeda dari algoritma sorting lainnya. Quick Sort membagi data menjadi beberapa bagian yang lebih kecil sehingga proses pengurutannya bisa lebih cepat. Karena itu, saya memilih Quick Sort untuk digunakan dalam mengurutkan playlist lagu berdasarkan durasi.

Kode program di atas digunakan untuk mengelola data playlist lagu dan mengurutkannya berdasarkan durasi menggunakan algoritma Quick Sort. Program memanfaatkan class Lagu_2511531007 sebagai objek untuk menyimpan informasi lagu yang terdiri dari judul, penyanyi, dan durasi. Setelah data lagu dimasukkan ke dalam array, program akan menampilkan data sebelum dan sesudah proses pengurutan sehingga pengguna dapat melihat hasil sorting secara langsung.

Di dalam class Sorting_2511531007 terdapat array dataLagu_1007 yang digunakan untuk menyimpan maksimal 20 objek lagu. Variabel jumlah_1007 digunakan untuk menyimpan jumlah data lagu yang saat ini terdapat dalam array. Pada method inputData_1007(), program mengisi array dengan tujuh data lagu yang terdiri dari judul, penyanyi, dan durasi lagu. Setelah seluruh data dimasukkan, variabel jumlah_1007 diisi dengan nilai 7 yang menunjukkan jumlah lagu yang tersedia.

Method tampilData_1007() berfungsi untuk menampilkan seluruh data lagu yang terdapat dalam array. Method ini menggunakan perulangan for untuk menampilkan setiap objek lagu melalui method toString_1007() yang terdapat pada class

Lagu_2511531007. Dengan demikian informasi lagu dapat ditampilkan dalam format yang lebih mudah dibaca.

Untuk mendukung proses Quick Sort, dibuat method `swap_1007()` yang berfungsi menukar posisi dua objek lagu dalam array. Method ini menggunakan variabel sementara `temp_1007` untuk menyimpan salah satu objek sebelum dilakukan pertukaran posisi.

Selain itu terdapat method `medianOfThree_1007()` yang digunakan untuk menentukan pivot menggunakan teknik median-of-three. Method ini membandingkan tiga elemen, yaitu elemen pertama (`low_1007`), elemen tengah (`mid_1007`), dan elemen terakhir (`high_1007`) berdasarkan durasi lagu. Setelah ketiga elemen diurutkan, nilai median dipindahkan ke posisi terakhir array dan digunakan sebagai pivot.

Proses pembagian data dilakukan pada method `partition_1007()`. Pada awal method, fungsi `medianOfThree_1007()` dipanggil untuk menentukan pivot. Nilai pivot kemudian diambil dari durasi lagu yang berada pada indeks terakhir. Selanjutnya dilakukan perulangan `for` untuk membandingkan durasi setiap lagu dengan nilai pivot. Jika durasi lagu lebih kecil dari pivot, maka posisi lagu tersebut akan ditukar menggunakan method `swap_1007()` sehingga seluruh lagu dengan durasi lebih kecil berada di sebelah kiri pivot. Setelah proses selesai, pivot ditempatkan pada posisi yang sesuai dan method mengembalikan indeks pivot yang baru.

Method `quickSort_1007(Lagu_2511531007[] arr_1007, int low_1007, int high_1007)` merupakan method utama algoritma Quick Sort yang menggunakan teknik rekursif. Jika indeks awal masih lebih kecil daripada indeks akhir, program akan memanggil method `partition_1007()` untuk menentukan posisi pivot. Setelah posisi pivot diperoleh, method `quickSort_1007()` akan dipanggil kembali untuk mengurutkan bagian kiri dan bagian kanan pivot hingga seluruh data tersusun secara terurut berdasarkan durasi.

Agar lebih mudah digunakan, dibuat method `quickSort_1007()` tanpa parameter yang berfungsi sebagai wrapper method. Method ini hanya perlu dipanggil satu kali dari luar class dan secara otomatis akan menjalankan proses Quick Sort pada seluruh data lagu yang terdapat dalam array.

Pada method `main()`, program diawali dengan membuat objek `playlist_1007` dari class `Sorting_2511531007`. Selanjutnya program menampilkan identitas praktikum dan pilihan algoritma yang digunakan, kemudian memanggil method

inputData_1007() untuk mengisi data lagu. Data lagu ditampilkan terlebih dahulu menggunakan method tampilData_1007() sebagai kondisi sebelum sorting. Setelah itu method quickSort_1007() dipanggil untuk melakukan pengurutan berdasarkan durasi lagu secara ascending. Setelah proses pengurutan selesai, program kembali menampilkan data lagu sehingga pengguna dapat melihat perubahan urutan lagu dari sebelum sorting menjadi sesudah sorting.

BAB III PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma Shell Sort, Quick Sort, dan Merge Sort dapat digunakan untuk mengurutkan data dengan baik dari nilai terkecil ke terbesar. Meskipun ketiga algoritma tersebut memiliki cara kerja yang berbeda, hasil yang diperoleh tetap sama yaitu data berhasil diurutkan sesuai urutannya.

Melalui praktikum ini, dapat dipahami bagaimana proses pengurutan data dilakukan oleh masing-masing algoritma. Selain itu, praktikum ini juga membantu dalam memahami implementasi algoritma sorting menggunakan bahasa pemrograman Java serta mengetahui perbedaan cara kerja antara Shell Sort, Quick Sort, dan Merge Sort.

3.2 Saran

1. Mahasiswa diharapkan dapat lebih sering berlatih mengerjakan soal atau membuat program secara mandiri, sehingga saat menghadapi ujian akhir praktikum tidak mengalami kesulitan dalam memahami maupun menyelesaikan soal yang diberikan.
2. Selain itu, mahasiswa sebaiknya tidak hanya mengikuti langkah praktikum, tetapi juga mencoba memahami alur program agar dapat mengerjakan tanpa bergantung pada contoh yang sudah ada.
3. Dengan adanya latihan yang cukup, diharapkan proses pembelajaran praktikum dapat menjadi lebih efektif dan hasil yang diperoleh mahasiswa menjadi lebih maksimal.

DAFTAR PUSTAKA

- [1] W. Wahyudi, *Struktur Data dengan Java*. CV Eureka Media Aksara, 2025. [Online]. Available: <https://books.google.co.id/books?id=EN2lEQAAQBAJ>.